

INFRASTRUCTURE BLUEPRINT

ScrapeSuite Hosting Guide

Production deployment strategies for the
Nuclear Fusion scraping engine

A comprehensive analysis of hosting providers, infrastructure requirements, cost projections, and deployment architectures for running ScrapeSuite at scale, covering 66,900 lines of code across 20 proxy modules with Playwright, BullMQ, PostgreSQL, Redis, and TOR.

1. Executive Summary

ScrapeSuite is a production-grade web scraping engine comprising 66,904 lines of TypeScript across 20 proxy modules, 50+ API endpoints, and a Nuclear Fusion self-replication architecture capable of managing 28.8 billion+ effective IPs across a 100-peer mesh. The system integrates Playwright headless browsers for JavaScript rendering, TOR circuits for anonymous routing, BullMQ job queues for distributed processing, PostgreSQL for persistent storage, and Redis for caching and session management. Hosting this system requires careful consideration of CPU, memory, storage, bandwidth, and networking constraints that go far beyond typical web application deployments.

This guide evaluates seven hosting strategies across three tiers: budget-friendly single-server deployments for development and low-volume production, mid-range clustered setups for growing workloads, and enterprise-grade distributed architectures for maximum throughput. Each option is assessed on cost-efficiency, scalability, latency, compliance, and operational complexity. The recommended path is a hybrid architecture combining Hetzner dedicated servers for heavy compute workloads with Fly.io for API gateway and auto-scaling, delivering the best balance of performance and cost at approximately \$200-400/month for a mid-scale deployment.

2. System Requirements Analysis

Before selecting a hosting provider, it is essential to understand the resource profile of each ScrapeSuite component. The system is not a monolith but rather a collection of specialized services with vastly different resource demands. Playwright browser instances are memory-hungry, each consuming 200-500MB of RAM. The TOR pool requires sustained bandwidth and low-latency network paths. The fusion core and chain reaction modules are CPU-intensive during proxy discovery and validation phases, while the API gateway itself is lightweight and benefits from edge deployment.

2.1 Component Resource Profile

Component	CPU	RAM	Disk	Network	Priority
API Gateway (Fastify)	Low (0.5 core)	256-512 MB	1 GB	Low	Edge/Any
Worker + Playwright	High (2-4 cores)	4-8 GB	5 GB + browsers	High (500 Mbps+)	Dedicated
Fusion Core + Chain Reaction	Medium (1-2 cores)	1-2 GB	500 MB	Medium	Dedicated
TOR Pool (50+ circuits)	Low-Medium	1-2 GB	1 GB	High (sustained)	Dedicated

BullMQ Workers	Low-Medium	512 MB-1 GB	Minimal	Medium	Any
PostgreSQL	Medium (1-2 cores)	2-4 GB	10-50 GB SSD	Low	Managed/Dedicated
Redis	Low (0.5 core)	1-4 GB	5 GB	Low	Managed/Any
CAPTCHA Solver	Medium (1 core)	512 MB-1 GB	Minimal	Medium (API calls)	Any
Web Unlocker	High (2 cores)	2-4 GB	2 GB	High	Dedicated
Free Proxy Discovery	Medium (1-2 cores)	1 GB	Minimal	High (55+ sources)	Any

Table 1: Per-component resource requirements for ScrapeSuite deployment

2.2 Aggregate Requirements by Scale

The total resource footprint depends heavily on concurrency targets. A minimal deployment handling 10 concurrent scrape jobs needs approximately 4 CPU cores, 8 GB RAM, and 50 GB SSD. A production deployment handling 100 concurrent jobs with full Nuclear Fusion replication active requires 16+ CPU cores, 32+ GB RAM, 200+ GB SSD, and at least 1 Gbps sustained bandwidth. At enterprise scale with 500+ concurrent jobs and the full 100-peer mesh, the system needs 64+ cores distributed across multiple machines, 128+ GB RAM, and 10 Gbps aggregate bandwidth.

Scale	Concurrent Jobs	CPU Cores	RAM	Storage	Bandwidth	Est. Cost/mo
Development	1-5	2-4	8 GB	50 GB SSD	100 Mbps	\$20-50
Staging / Small	5-20	4-8	16 GB	100 GB SSD	500 Mbps	\$50-150
Production (Medium)	20-100	16+	32 GB	200 GB NVMe	1 Gbps	\$150-400
Production (Large)	100-500	32-64	64-128 GB	500 GB NVMe	1-10 Gbps	\$400-1200
Enterprise	500+	64+ (cluster)	128+ GB	1 TB+ NVMe	10 Gbps+	\$1200+

Table 2: Aggregate resource requirements and estimated monthly costs by deployment scale

3. Hosting Provider Deep Dive

This section evaluates the leading hosting providers suitable for ScrapeSuite, organized by category. Each provider is assessed on raw performance per dollar, scalability model, geographic availability, compliance posture, and suitability for the specific demands of a high-throughput scraping engine with browser automation.

3.1 Hetzner — Best Price-to-Performance (Recommended)

Hetzner is the undisputed leader in price-to-performance ratio for dedicated servers in Europe. Based in Germany, they operate data centers in Falkenstein, Nuremberg, and Helsinki, with recent expansion into Ashburn, Virginia (US). For ScrapeSuite, Hetzner dedicated servers offer the most RAM and CPU cores per dollar of any provider on the market, which is critical given that Playwright browser instances are the primary resource bottleneck. The EX44 server at approximately 44 EUR/month delivers an Intel i5-13500 with 14 cores (6P+8E), 64 GB DDR5 RAM, and 2x 512 GB NVMe SSD, providing enough headroom to run 15-20 concurrent browser sessions alongside the fusion core, TOR pool, and database services. The AX102 at approximately 50 EUR/month offers an AMD Ryzen 9 7950X with 16 cores, 128 GB DDR5, and 2x 1 TB NVMe, which can handle 50+ concurrent sessions with room to spare.

The primary caveat with Hetzner is their recent 20-40% price increases (effective June 2026) and aggressive terms of service enforcement. There are documented cases of servers being removed with short notice for ToS violations, which is particularly relevant for scraping workloads. Hetzner also charges significant setup fees (39-269 EUR), though these are one-time costs. Bandwidth is generous at 1 Gbps unmetered on dedicated servers, with 20 TB/month included on cloud VPS. IPv4 addresses cost extra at 0.84 EUR/month each, and /29 subnets (6 usable IPs) run 22 EUR/month. For a scraping engine that needs multiple IP addresses for proxy rotation, these additional IP costs should be factored in.

Recommended Hetzner Configurations

Server	CPU	RAM	Storage	Bandwidth	Price/month	Best For
EX44	Intel i5-13500 (14C)	64 GB DDR5	2x 512GB NVMe	1 Gbps unmetered	~44 EUR	Small-Medium prod
AX102	Ryzen 9 7950X (16C)	128 GB DDR5	2x 1TB NVMe	1 Gbps unmetered	~50 EUR	Medium-Large prod
AX42	Ryzen 5 3600 (6C)	64 GB DDR4	2x 512GB NVMe	1 Gbps unmetered	~44 EUR	Budget production
CCX33 (Cloud)	8 vCPU (ARM)	32 GB	320 GB SSD	20 TB included	~38 EUR	API + lightweight

Table 3: Hetzner server configurations recommended for ScrapeSuite

3.2 OVHcloud — Best for Unlimited Bandwidth & Anti-DDoS

OVHcloud, headquartered in France, is the largest hosting provider in Europe and operates data centers across four continents. Their key differentiator for ScrapeSuite is unlimited bandwidth on all dedicated servers, with permanent anti-DDoS protection included at no extra cost. This is significant because scraping workloads can generate massive outbound traffic (each Playwright session downloads 2-10 MB of resources), and DDoS retaliation from target websites is a real operational risk. OVH also offers the widest geographic footprint with locations in France, Canada, Singapore, Australia, India, and the US, enabling low-latency scraping from regions closer to target servers.

The Advance server line, built on AMD EPYC 4004/4005 processors, offers up to 384 cores and 3 TB of DDR5 memory on the 2026 generation. The Advance-1 at approximately 80 EUR/month provides an AMD EPYC 4210P (6C/12T), 32 GB DDR5, and 2x 480 GB NVMe with 1 Gbps unmetered bandwidth. The Advance-3 at approximately 160 EUR/month steps up to an AMD EPYC 8324P (32C/64T), 128 GB DDR5, and 2x 960 GB NVMe. While OVH pricing is higher than Hetzner for equivalent specs, the unlimited bandwidth, integrated anti-DDoS, and global presence make it compelling for high-traffic scraping operations. OVH also has more lenient ToS enforcement regarding scraping compared to Hetzner, reducing the risk of sudden server termination.

Server	CPU	RAM	Storage	Bandwidth	Price/mo
Advance-1	EPYC 4210P (6C/12T)	32 GB DDR5	2x 480GB NVMe	1 Gbps unmetered	~80 EUR
Advance-2	EPYC 6314P (8C/16T)	64 GB DDR5	2x 480GB NVMe	1 Gbps unmetered	~120 EUR
Advance-3	EPYC 8324P (32C/64T)	128 GB DDR5	2x 960GB NVMe	1 Gbps unmetered	~160 EUR
Rise-1	Intel E-2388G (8C/16T)	32 GB DDR4	2x 512GB NVMe	500 Mbps unmetered	~60 EUR

Table 4: OVHcloud dedicated server configurations for ScrapeSuite

3.3 Fly.io — Best for Auto-Scaling & Edge Deployment

Fly.io is a container-native platform that runs Docker images on lightweight VMs (Machines) across 30+ global regions. ScrapeSuite already has a Fly.io configuration (fly.toml) with separate API and worker process groups, making it the path of least resistance for initial deployment. Machines boot in under 500ms, enabling true auto-scaling where worker instances spin up on demand during traffic spikes and scale to zero during quiet periods. This is particularly valuable for scraping workloads that tend to be bursty, with peak demand 5-10x above average.

However, Fly.io has significant cost limitations at scale. A single 8-CPU, 16 GB Machine costs approximately \$29/month on the Standard plan, but running 50+ concurrent Playwright sessions requires at least 4 such machines (\$116/month) plus a 32 GB PostgreSQL instance (\$962/month at

performance-4x tier), Redis (\$15-50/month), and bandwidth charges. Total production costs easily exceed \$500-800/month for a medium-scale deployment, roughly 2-3x the cost of equivalent Hetzner hardware. The free tier was eliminated in 2024, with new signups receiving only a 2-hour trial. Fly.io also requires Docker expertise and has a steeper learning curve than traditional VPS providers. The platform excels as an API gateway and for lightweight services, but the heavy-lifting workers should run on dedicated hardware for cost efficiency.

Resource	Specification	Monthly Cost	Notes
Machine (1 CPU, 256 MB)	Shared CPU	~\$1.94	API gateway only
Machine (4 CPU, 8 GB)	Shared CPU	~\$23.04	Light worker
Machine (8 CPU, 16 GB)	Dedicated CPU	~\$29.00+	Heavy worker + Playwright
Fly Postgres (Performance-2x)	2 CPU, 8 GB	~\$73/month	Managed HA database
Fly Postgres (Performance-4x)	4 CPU, 32 GB	~\$962/month	Enterprise DB tier
Redis	1 GB baseline	~\$15-50/month	Managed Redis
Bandwidth	First 160 GB free	\$0.10/GB after	Scraping = heavy egress
Volume storage	3 GB free	\$0.15/GB/month	Persistent data

Table 5: Fly.io pricing breakdown for ScrapeSuite components

3.4 Vultr — Best for Global Reach & Hourly Billing

Vultr offers cloud compute and bare metal servers across 32 global locations, more than any other budget provider. Their hourly billing model (as low as \$0.004/hour for basic instances) makes Vultr ideal for development, testing, and burst-capacity scenarios where servers can be spun up for a few hours and destroyed. Bare metal servers start at \$150/month for an 8-core Intel E-2388G with 64 GB RAM, which is competitive with OVH but significantly more expensive than Hetzner for equivalent specs. Vultr also offers High Frequency Compute instances with AMD EPYC and NVMe storage, starting at \$6/month for 1 CPU and 1 GB RAM, suitable for running lightweight ScrapeSuite components like the API gateway, CAPTCHA solver relay, or Redis cache.

The key advantage of Vultr is flexibility: 32 locations mean you can deploy scraping workers close to target websites geographically, reducing latency and avoiding region-based blocking. Their Snapshot and Custom ISO features allow rapid provisioning of pre-configured ScrapeSuite images. However, Vultr lacks managed PostgreSQL and Redis offerings (you must self-host), and their bandwidth is not truly unlimited, with overage charges applying on some plans. For budget-conscious operators, Vultr works best as a complement to Hetzner rather than a primary provider.

3.5 AWS / GCP — Enterprise Scale Only

Amazon Web Services and Google Cloud Platform are overkill for most ScrapeSuite deployments but become necessary at enterprise scale (500+ concurrent jobs) or when compliance requirements (SOC 2, HIPAA, GDPR data residency) mandate certified infrastructure. AWS c6i.4xlarge (16 vCPU, 32 GB) instances cost approximately \$0.612/hour on-demand (\$446/month) or \$0.204/hour with a 1-year reserved instance (\$149/month). Google Cloud C2 (16 vCPU, 64 GB) instances are similarly priced. Both offer managed PostgreSQL (RDS / Cloud SQL) and Redis (ElastiCache / Memorystore) that eliminate database operational overhead.

The critical downside is egress bandwidth costs. AWS charges \$0.09/GB for the first 10 TB of egress, and GCP charges \$0.105/GB. A medium-scale scraping operation moving 5 TB of data per month would incur \$450+ in bandwidth alone on AWS, compared to zero on Hetzner or OVH. For this reason, AWS and GCP are only recommended when specific compliance certifications are required, when using managed AI/ML services for advanced CAPTCHA solving, or when operating at enterprise scale where cost efficiency is secondary to reliability and compliance. A hybrid approach using AWS for the API layer and managed databases while offloading heavy compute to Hetzner is often the most cost-effective enterprise architecture.

3.6 Railway / Render — Development & Staging Only

Railway and Render are Heroku-alternative PaaS platforms optimized for developer experience. Both support Docker deployments, offer managed PostgreSQL and Redis, and provide zero-configuration scaling. Railway excels with instant deploys and a clean CLI, while Render offers more configuration depth and native runtime support for Node.js. However, both platforms are significantly more expensive than bare metal providers at equivalent specs, with a 4 vCPU, 8 GB instance running approximately \$40/month on Render, roughly five times the cost of a similar Hetzner cloud VPS. Neither platform supports Playwright browser installation easily due to missing system dependencies, and both have strict memory limits that make running Chromium instances impractical. These platforms are recommended only for hosting the ScrapeSuite API gateway in development/staging environments, not for production worker processes.

4. Recommended Deployment Architectures

Based on the analysis above, three architectures are recommended depending on scale and budget. All three follow the principle of separating the API gateway from heavy compute workers, and using managed services for databases where cost-effective. The Nuclear Fusion architecture of ScrapeSuite, with its 5-second reactor loops and adaptive rate controls, requires low-latency communication between the fusion core and worker processes, which influences the network topology of each design.

4.1 Architecture A: Budget Single-Server (Solo Operator)

The simplest deployment runs everything on a single Hetzner AX42 or EX44 dedicated server. This architecture is suitable for individual operators or small teams running up to 20 concurrent scrape jobs. The server runs Docker Compose with separate containers for the API, worker, PostgreSQL, and Redis, all sharing the same host. The Playwright browser pool is limited to 10-15 concurrent instances given the 64 GB RAM constraint (each instance consumes 200-500 MB). The TOR pool operates with 20-30 circuits instead of the full 50+. This setup costs approximately 44-50 EUR/month plus 10-20 EUR for additional IPv4 addresses, delivering a total cost of ownership under \$75/month.

Pros: Simplest to deploy and maintain. Single SSH session for debugging. No network latency between components. Lowest possible cost. **Cons:** No redundancy. Single point of failure. Limited scalability. Cannot run the full Nuclear Fusion mesh. Playwright concurrency is capped by available RAM. TOR pool throughput is limited. Database performance degrades under load because PostgreSQL competes with workers for CPU and I/O.

4.2 Architecture B: Hybrid Hetzner + Fly.io (Recommended)

This is the recommended architecture for production deployments. It combines Hetzner dedicated servers for compute-heavy workers with Fly.io for the API gateway and auto-scaling surge capacity. The Hetzner AX102 (128 GB RAM, 16 cores) serves as the primary worker node, running Playwright instances, TOR pool, fusion core, and chain reaction engine. PostgreSQL runs as a managed Fly Postgres instance (Performance-2x at \$73/month), eliminating database administration overhead. Redis runs on a 2 GB Fly instance (\$10/month). The API gateway runs on a Fly.io Machine that auto-scales from 1 to 4 instances based on request volume.

During traffic spikes, Fly.io worker Machines spin up automatically to handle overflow scraping jobs, using the shared PostgreSQL and Redis instances. These surge workers are more expensive per-job than the Hetzner dedicated worker but only run for minutes at a time, keeping costs manageable. The fusion core on the Hetzner server can connect to Fly.io workers via Fly.io private networking (6wire), enabling the nuclear replication system to distribute proxy validation and discovery tasks across both platforms. Total cost: approximately \$150-300/month for a deployment handling 50-100 concurrent jobs, with auto-scaling headroom for 200+ jobs during peaks.

Component	Platform	Spec	Cost/month
Primary Worker	Hetzner AX102	16C/128GB/2x1TB NVMe	~50 EUR
API Gateway	Fly.io Machine (2 CPU, 4 GB)	Auto-scale 1-4x	~\$15-60
Surge Workers	Fly.io Machine (4 CPU, 8 GB)	Scale-to-zero	~\$0-100 (burst)

PostgreSQL	Fly Postgres Performance-2x	2 CPU, 8 GB, HA	~\$73
Redis	Fly.io (2 GB)	Managed	~\$10
Additional IPv4 (Hetzner)	/29 subnet (6 IPs)	For proxy rotation	~22 EUR
Bandwidth (Fly.io)	~200 GB/month	Egress charges	~\$4
		TOTAL	~\$175-300

Table 6: Architecture B component costs (Hybrid Hetzner + Fly.io)

4.3 Architecture C: Multi-Server Cluster (Enterprise)

For large-scale operations handling 500+ concurrent jobs with the full Nuclear Fusion 100-peer mesh, a multi-server cluster is required. This architecture distributes ScrapeSuite components across dedicated servers with a load balancer, shared storage, and database clustering. The core cluster consists of three Hetzner AX102 servers (384 GB total RAM, 48 cores) running behind a HAProxy load balancer. Two servers run Playwright workers, TOR pool, and fusion modules, while the third serves as a dedicated PostgreSQL primary with streaming replication to a standby. Redis runs in sentinel mode across all three servers for HA. An OVH server in a different geographic region provides DR (disaster recovery) and proxy diversity, with its own TOR exit nodes and free proxy discovery sources targeting region-specific proxy lists.

The API gateway runs on Fly.io with global edge deployment for low-latency client access. A dedicated Hetzner storage server (SX64 at approximately 79 EUR/month, 2x 6 TB HDD) provides shared storage for Playwright browser caches, session data, and scraped content archives. Monitoring runs on a lightweight Vultr instance (\$6/month) with Prometheus + Grafana. Total cluster cost is approximately \$400-800/month, scalable to \$1200+/month by adding worker nodes. This architecture supports the full Nuclear Fusion system with critical mass at 5,000+ proxies, cascade depth of 8, and 5-second reactor loops across the entire mesh.

Component	Platform	Spec	Cost/month
Worker Node 1	Hetzner AX102	16C/128GB/2x1TB NVMe	~50 EUR
Worker Node 2	Hetzner AX102	16C/128GB/2x1TB NVMe	~50 EUR
DB Primary	Hetzner AX102	16C/128GB/2x1TB NVMe	~50 EUR
DR Node	OVH Advance-1	6C/32GB/2x480GB NVMe	~80 EUR
Storage Server	Hetzner SX64	2x6TB HDD	~79 EUR

API Gateway	Fly.io (edge, 4 regions)	2 CPU, 4 GB, auto-scale	~\$60
Monitoring	Vultr (1 CPU, 1 GB)	Prometheus + Grafana	~\$6
Load Balancer	Hetzner LB	Managed	~8 EUR
		TOTAL	~\$440-520

Table 7: Architecture C component costs (Enterprise multi-server cluster)

5. Provider Cost Comparison

The following table provides a side-by-side comparison of all evaluated providers for a medium-scale ScrapeSuite deployment (approximately 50 concurrent scrape jobs, 16 CPU cores, 32 GB RAM, managed PostgreSQL and Redis). Costs include compute, database, storage, bandwidth, and additional IP addresses where applicable. All prices are normalized to monthly USD for easy comparison.

Provider	Compute	Database	Bandwidth	IP Addresses	Total/mo	Rating
Hetzner (Dedicated)	\$50-55	Self-hosted	\$0 (unmetered)	\$5-25	\$55-80	Best Value
OVHcloud	\$85-130	Self-hosted	\$0 (unmetered)	Included	\$85-130	Best BW
Fly.io (Full Stack)	\$60-120	\$73-962	\$10-50	Included	\$150-1100	Easiest
Vultr (Bare Metal)	\$150+	Self-hosted	\$0-20	\$3-12	\$150-180	Most Regions
AWS (Reserved)	\$150-300	\$50-150	\$200-450	\$3-5	\$400-900	Compliance
GCP (Committed)	\$140-280	\$50-120	\$200-500	\$3-5	\$400-900	Compliance
Hybrid (Hetz+Fly)	\$50-80 + \$15-60	\$73	\$4-10	\$5-25	\$150-250	Recommended

Table 8: Provider cost comparison for medium-scale deployment (50 concurrent jobs)

6. Step-by-Step Deployment Guide

This section provides a practical deployment walkthrough for the recommended Architecture B (Hybrid Hetzner + Fly.io). The process is divided into four phases: server provisioning, Docker configuration, Fly.io deployment, and production hardening. Each phase includes specific commands and

configuration snippets tailored for ScrapeSuite.

6.1 Phase 1: Hetzner Server Provisioning

Order a Hetzner AX102 dedicated server from the Hetzner Robot panel (robot.hetzner.com). Select the Falkenstein (fsn) data center for lowest latency to Western Europe and North America. Install Ubuntu 24.04 LTS via the Hetzner installimage tool. After the server is online, SSH in and run the following setup commands. First, install Docker and Docker Compose: update the package index, install prerequisite packages, add the Docker GPG key, add the Docker repository, and install docker-ce, docker-ce-cli, and containerd.io. Then install Playwright system dependencies including libnss3, libnspr4, libatk1.0-0, libatk-bridge2.0-0, libcups2, libdrm2, libxkbcommon0, libxcomposite1, libxdamage1, libxrandr2, libgbm1, libpango-1.0-0, libcairo2, libasound2, and fonts-liberation. Clone the ScrapeSuite repository and configure environment variables for DATABASE_URL (pointing to Fly Postgres), REDIS_URL (pointing to Fly Redis), and all API keys for CAPTCHA providers and proxy services.

6.2 Phase 2: Docker Compose Configuration

ScrapeSuite includes a production-ready docker-compose.yml that defines four services: the API gateway, worker process, PostgreSQL, and Redis. For the hybrid architecture, modify the compose file to remove the PostgreSQL and Redis services (these will run on Fly.io) and update the environment variables to point to the Fly.io managed instances. The worker service should set shm_size to 4 GB (required for Chromium) and limit CPU and memory to leave headroom for the OS. Set HTTP_WORKER_CONCURRENCY to 50 and BROWSER_WORKER_CONCURRENCY to 15 on a 128 GB RAM server, adjusting downward if running on a 64 GB server. The API service should expose port 3001 and configure health checks against the /health endpoint. Run "docker compose up -d" to start all services, then verify with "docker compose logs -f" that the fusion core initializes correctly and begins the 5-second reactor loops.

6.3 Phase 3: Fly.io Deployment

Install the Fly.io CLI (flyctl) and authenticate with "flyctl auth login". Create a new Fly app with "flyctl apps create scrapesuite-api" and set the primary region to lhr (London) for proximity to the Hetzner Falkenstein data center. Deploy the API gateway using the existing Dockerfile and fly.toml configuration. Provision a Fly Postgres database with "flyctl postgres create" selecting the Performance-2x plan (2 CPU, 8 GB) for medium-scale deployments. Attach the database to the API app with "flyctl postgres attach". Similarly, create a Redis instance with "flyctl redis create" selecting a 2 GB plan. Configure the worker Machine process group to auto-scale based on CPU utilization, with a minimum of 0 machines (scale-to-zero) and a maximum of 4 machines (16 CPU, 32 GB total surge capacity). Set the scale-to-zero idle timeout to 120 seconds to avoid premature shutdown during brief quiet periods between scraping bursts.

6.4 Phase 4: Production Hardening

Production hardening covers seven critical areas. First, network security: configure the Hetzner firewall to allow inbound traffic only on ports 22 (SSH), 3001 (API), and 5432 (PostgreSQL from Fly.io IPs only). Enable fail2ban with aggressive SSH settings (max 3 retries, 1-hour ban). Second, monitoring: deploy Prometheus and Grafana on a lightweight Vultr instance, configure Node Exporter on the Hetzner server, and set up alerts for CPU usage above 85%, RAM usage above 90%, and disk usage above 80%. Third, logging: configure Docker logging drivers to send logs to a centralized Loki instance, with retention policies of 7 days for debug logs and 30 days for error logs. Fourth, backup: set up daily PostgreSQL dumps to Hetzner Storage Box (costs 3.81 EUR/month for 100 GB), with 30-day retention and weekly full backups plus daily incrementals. Fifth, SSL/TLS: terminate TLS at the Fly.io edge with automatic Let's Encrypt certificates, and use Fly.io private networking (6wire) for all internal communication between Fly.io and Hetzner. Sixth, secret management: store all API keys, database credentials, and JWT secrets in Fly.io secrets (encrypted at rest), never in environment variables or .env files. Seventh, rate limiting: configure the Fastify rate limiter to allow 100 requests per minute per IP for public endpoints and 1000 per minute for authenticated endpoints, with a hard limit of 5000 per minute to prevent abuse.

7. Scaling Strategy

ScrapeSuite is designed to scale horizontally through its Nuclear Fusion architecture. The scaling strategy follows a three-phase progression: vertical scaling on a single server, horizontal scaling across multiple workers, and mesh scaling across geographic regions. Each phase unlocks additional capabilities of the fusion system while maintaining the 5-second reactor loop cadence and adaptive rate controls that make the system responsive to changing conditions.

7.1 Phase 1: Vertical Scaling (Single Server)

Start with a single Hetzner AX102 and maximize its utilization before adding more servers. The 128 GB RAM supports approximately 50 concurrent Playwright sessions (at 400 MB average per session, leaving 8 GB for PostgreSQL, Redis, and OS). Monitor RAM utilization via the ScrapeSuite dashboard at `/api/v1/proxy/stats/unified`. When average RAM usage exceeds 85% during peak hours, or when the BullMQ queue backlog consistently exceeds 100 pending jobs, it is time to move to Phase 2. The fusion core should be configured with a critical mass threshold of 2,000 proxies and cascade depth of 6 at this stage, with the afterburner mode disabled to conserve resources.

7.2 Phase 2: Horizontal Scaling (Multiple Workers)

Add a second Hetzner AX102 as a dedicated worker node. The first server becomes the "control plane" running the API gateway, fusion core, PostgreSQL, and Redis, while the second server runs exclusively as a Playwright and TOR worker. This separation eliminates resource contention between the fusion core and browser instances. Connect the workers via a private VPN (WireGuard recommended, adds only 2-3ms latency). Configure the BullMQ worker on the second server to connect to Redis on the control plane via the VPN tunnel. The fusion core on the control plane distributes proxy validation tasks evenly across both servers using the quantum tunnel module. At this stage, enable the fusion afterburner and set critical mass to 5,000 proxies with cascade depth 8. Total capacity: approximately 100 concurrent jobs across both servers.

7.3 Phase 3: Mesh Scaling (Geographic Distribution)

Deploy additional worker nodes in different geographic regions using OVH (France/Canada/Singapore) and Vultr (32 locations). Each regional node runs a lightweight worker process that connects back to the central fusion core via WireGuard mesh. The Nuclear Fusion system treats each node as a "peer" in the mesh, sharing validated proxies, TOR circuits, and free proxy discoveries across all nodes in real-time. With 10 peers, the effective IP pool expands from 288 million to 2.88 billion addresses. With 100 peers, it reaches 28.8 billion. The chain reaction cascade propagates across the mesh with sub-second latency, and the neutron multiplier effect means that successful proxy discoveries on one node automatically trigger discovery cascades on all other nodes. Monitor mesh health via the `/api/v1/proxy/fusion/stats` endpoint, which provides per-peer latency, proxy counts, and cascade rates.

8. Security and Compliance Considerations

Hosting a scraping engine introduces unique security challenges beyond standard web application concerns. The system handles sensitive API keys for CAPTCHA providers, proxy service credentials, TOR circuit identifiers, and potentially user data from scraped content. Additionally, the act of web scraping itself can trigger retaliatory measures from target websites, including DDoS attacks, IP blacklisting, and legal challenges. This section addresses these concerns with practical mitigation strategies.

8.1 Data Security

All API keys (2Captcha, Anti-Captcha, CapMonster, CapSolver, Bright Data) must be stored in encrypted secrets management, never in environment variables or configuration files committed to version control. Use Fly.io secrets for Fly-deployed components and HashiCorp Vault (or a simpler alternative like Docker Secrets) for Hetzner-deployed components. Database connections must use TLS (Fly Postgres enables this by default; for self-hosted PostgreSQL on Hetzner, configure `ssl=on` in `postgresql.conf` and require SSL for all connections). Encrypt all scraped data at rest using LUKS disk encryption on Hetzner NVMe drives. Rotate the GitHub Personal Access Token immediately if it has

been exposed in any configuration file or commit history, as compromised tokens can grant full repository access.

8.2 Network Security

Deploy OVH anti-DDoS (included free with OVH servers) or Cloudflare Spectrum (starting at \$20/month) on the API gateway to absorb volumetric attacks. Use Hetzner firewall rules to restrict SSH access to known IP addresses only. Configure the WireGuard VPN mesh with a dedicated private subnet (10.13.0.0/16) that is completely isolated from the public internet. Enable Fly.io private networking (6wire) for all communication between Fly.io apps and the Hetzner worker via a Fly.io WireGuard gateway. Set strict outbound firewall rules on the Hetzner server to prevent compromised containers from exfiltrating data to unauthorized endpoints. Use fail2ban with aggressive settings and install the CrowdSec collaborative intrusion detection system for real-time threat intelligence sharing.

8.3 Legal Compliance

Web scraping operates in a legal gray area that varies by jurisdiction. In the EU, the GDPR applies to any personal data scraped from websites, requiring a lawful basis for processing and data minimization. The EU Database Directive protects against unauthorized extraction of substantial parts of databases. In the US, the CFAA (Computer Fraud and Abuse Act) has been narrowed by the *hiQ vs. LinkedIn* Supreme Court decision, but still applies to circumventing authentication. Host ScrapeSuite in a jurisdiction with favorable legal frameworks: Germany (strong privacy laws, Hetzner/OVH based there) for EU operations, or Virginia (US, favorable CFAA interpretation) for US operations. Implement robust robots.txt compliance checking in the Web Unlocker module and provide opt-out mechanisms for website operators who request exclusion from scraping. Consult a technology attorney before deploying at enterprise scale.

9. Final Recommendation

For most ScrapeSuite operators, the Hybrid Hetzner + Fly.io architecture (Architecture B) delivers the best combination of cost-efficiency, scalability, and operational simplicity. The Hetzner AX102 at approximately 50 EUR/month provides 128 GB RAM and 16 cores, enough to run 50+ concurrent Playwright sessions with the full Nuclear Fusion system active. Fly.io handles the API gateway, managed PostgreSQL, and auto-scaling surge capacity, eliminating database administration overhead while adding elasticity for traffic spikes. Total cost of approximately \$175-300/month is roughly one-third the cost of an equivalent AWS deployment and one-half the cost of running entirely on Fly.io.

Start with Architecture A (single Hetzner server) for development and initial production, then migrate to Architecture B when you need managed databases and auto-scaling. Scale to Architecture C (multi-server cluster) when concurrent jobs exceed 100 or when you need geographic distribution for the

Nuclear Fusion mesh. Regardless of architecture, always separate the API gateway from heavy compute workers, use managed databases where cost-effective, and implement the security measures outlined in Section 8 before exposing the system to the public internet.

Architecture	Scale	Cost/mo	Complexity	Redundancy	Best For
A: Single Server	1-20 jobs	\$55-80	Low	None	Solo dev/staging
B: Hybrid Hetz+Fly	20-100 jobs	\$175-300	Medium	Partial	Recommended production
C: Multi-Server	100-500+ jobs	\$440-1200	High	Full HA	Enterprise/large scale

Table 9: Architecture comparison summary